TECHNICAL ROADMAP

TEAM INFORMATION

Team Name	UNSRAT - TouLearn
Institution	Universitas Sam Ratulangi
Country	Indonesia
Track	AI for Education
Team Leader Name	Ahmad Triadi Julianto M, triadim.works@gmail.com, +6282293277501

SECTION 1: EXECUTIVE SUMMARY (PROBLEM-SOLUTION FIT)

The ASEAN region is confronting an acute numeracy crisis; average achievement scores lag more than 100 points below the OECD global standard (472), with 81.7% of students performing below the baseline proficiency level. The 11–15 age cohort represents the most critical point of cognitive vulnerability, as the transition toward abstract mathematical curricula during this phase empirically triggers an academic performance decline of up to 0.29 standard deviations. Conventional rote learning triggers cognitive overload and math anxiety, significantly reducing active student participation. Meanwhile, current AI educational innovations exhibit structural weaknesses: single models are prone to hallucinations, and existing multi-agent systems rely heavily on static text, failing to provide crucial spatial mathematical representations. Consequently, there is an urgent need to integrate multi-agent architectures with interactive visualization to bridge this pedagogical gap and effectively reduce student anxiety.

Socratix addresses these deficiencies through a Multi-Agent AI framework grounded in Socratic pedagogy, shown to outperform standard chatbots in developing critical thinking. Specialized agents handle intent classification, problem analysis, and response generation, while a deterministic validator verifies mathematical correctness, minimizing hallucination risks. Real-time visualization animates mathematical objects per Socratic step, and gamification through an evolving geometric character reduces math anxiety, fostering numeracy competencies among ASEAN adolescent learners.

SECTION 2: TECHNICAL ARCHITECTURE

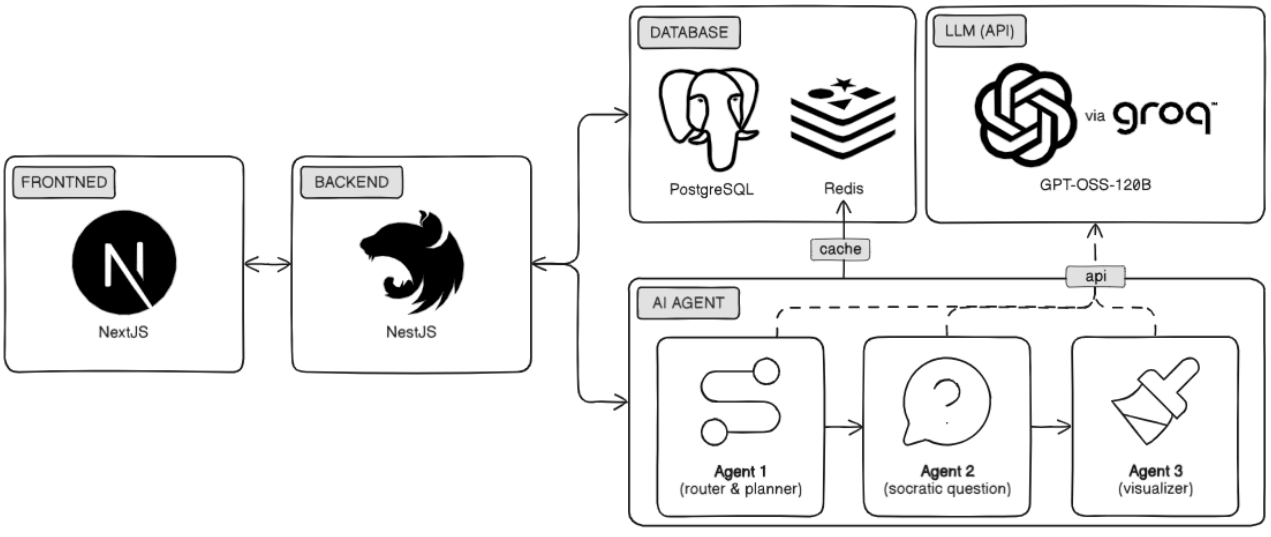
2.1 System Components:

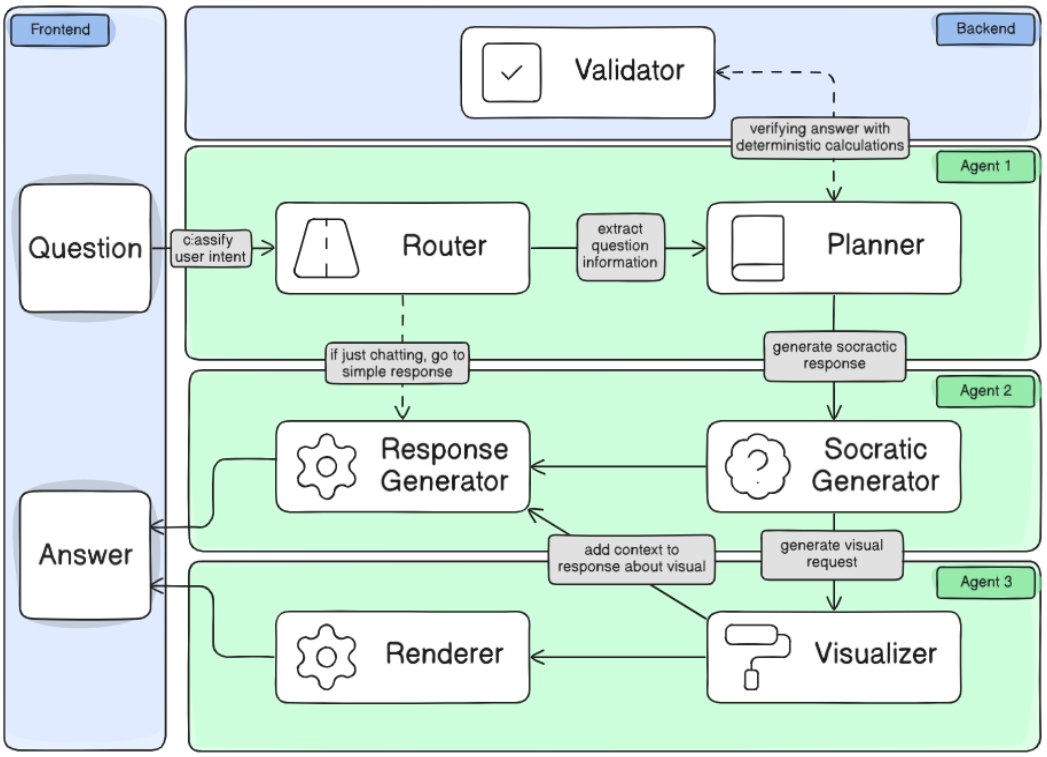
- **Inputs:** Socratix will receive student input through the chat interface, including math questions, problem statements, attempted answers, step-by-step reasoning, or math expressions such as ¾, x + 3 = 7, and (2,3). These inputs may cover topics such as fractions, algebra, geometry,

ratios, angles, charts, or coordinate planes, and they are passed into the multi-agent pipeline so the system can understand the problem, extract key information, and determine the appropriate tutoring response.

- **Processing Core:** The AI Processing happens in a multi-agent pipeline using an LLM through Groq API. The flow works as follows:
    - *First agent (planner, classifier, router)*: The first agent analyzes the user input. It validates whether the input is a math-related problem, classifies the students intent, routes the request to the correct learning flow, plans the next step, and extracts important mathematical information such as equations, numbers, variables, topics and known values.
    - *Second agent (socratic question generator)*: The second agent uses the extracted information and learning plan to generate a socratic question. This question is designed to guide the student toward the next reasoning step without directly giving the final answer. At the same time, this agent creates a visualization request based on the socratic question, so the system knows that visual support is needed.
    - *Third agent (visualization generator)*: The third agent receives the visualization request and generates a structured visualization plan. It maps the mathematical object involved in the problem, chooses the most suitable visualization type, and prepares a *scene plan* that matches the socratic questions. For example, it may choose a bar for fractions, a balance scale for equations, a number line for integers, or a coordinate plane for graph related problems.
- **Outputs:** The final output delivered to the user is an interactive learning experience that includes a socratic guiding questions, hints or feedback based on the student's reasoning, validation of the student's answer or learning step and an interactive visual representation of the math concept when needed. For example, if the students ask about "what is the value of ¾", Socratic may guide them with a question about the denominator while showing a fraction bar divided for equal parts with three parts highlighted.

2.2 Architecture Diagram:





SECTION 3: AI APPROACH & MODEL SELECTION

- **Primary AI Approach:** Multi Agentic Generative AI. Socratix uses a Multi-Agentic Generative AI approach, where the tutoring system is divided into multiple specialized agents rather than relying on a single general-purpose model. The system includes a Router Agent, Planner Agent, Prompt Builder, Dynamic Visualizer Agent, and Response Generator Agent, supported by a deterministic Math Validator. Each component has a focused responsibility. The Router identifies the student's intent, the Planner extracts the problem context, the Validator checks mathematical correctness, the Visualizer generates interactive visual representations, and the Response Generator produces Socratic tutoring responses. This structure makes the system more reliable, explainable, and suitable for math education.
- **Model Selection:** gpt-oss-120b via Groq API.
- **Reasoning:** Using gpt-oss-120b via Groq API is efficient for Socratix because the system requires both strong reasoning capability and fast response time. In an interactive tutoring environment, students need quick feedback after submitting an answer. If the AI takes too long to respond, the learning flow can be disrupted and students may lose engagement. Groq API helps reduce inference latency, making it more suitable for real-time Socratic dialogue and dynamic visualization. The gpt-oss-120b model is also suitable because it is an open-weight reasoning model that can support complex language understanding, structured problem analysis, and step-by-step educational response generation. This is important for Socratix because the AI must not only answer questions, but also understand the student's reasoning, identify misconceptions, and guide them through reflective questions. The multi-agent architecture further improves efficiency because not every task is handled by the model in the same way. Simple routing can be handled first through regex and keyword classification, while

gpt-oss-120b is only used when deeper reasoning or fallback classification is needed. This reduces unnecessary model calls and improves response speed. Mathematical correctness is handled by a deterministic Math.js validator. This means the LLM is not the only source of truth for math answers. The validator checks whether the student's answer is correct before the Response Generator produces feedback, reducing hallucination risk and improving reliability.

SECTION 4: DATA STRATEGY & ETHICS

- **Data Sources:** Socratix will use custom data collection from user interactions within the application, including math problems, student answers, conversation history, error patterns, validation results, and visualization outputs. This data is relevant because it is generated directly from the core learning pipeline: Router, Planner, Validator, Prompt Builder, Visualizer, and Response Generator.
- **Data Quality & Cleaning:** Data quality will be maintained through structured validation at each stage of the pipeline. The Router classifies user intent, the Planner extracts problem context, the Validator checks answers deterministically using a math engine, and the State Manager stores session history consistently. Incomplete, ambiguous, duplicated, or invalid data will be flagged before being used for evaluation or system improvement.
- **Licensing & Legality:** Since Socratix uses custom-collected data from user interactions, it does not depend on external dataset licenses such as MIT or Creative Commons. However, all data collection and usage will follow user consent, privacy policies, and hackathon rules. To strengthen data governance, Socratix will align its data handling procedures with recognized standards such as ISO/IEC 27001 for information security management, ISO/IEC 27701 for privacy information management, and ISO/IEC 42001 for responsible AI management. This ensures that user data is collected, stored, processed, and evaluated responsibly, securely, and transparently.
- **Bias Mitigation & Fairness:** Bias will be reduced by testing the system with diverse inputs, including different student ability levels, language styles, math problem types, and error patterns. The system will not be optimized only for one type of user, language expression, or answer pattern. The intent-based pipeline, deterministic math validation, and Socratic prompting help keep responses focused on the student's learning process rather than assumptions about the student's background.

SECTION 5: DEVELOPMENT MILESTONES (AGILE ROADMAP)

Phase	Activity / Task	Tools Used	Expected Outcome
Sprint 1 (April 21 - May 3)	Foundation & Backend Readiness	Next.js, NestJS, Prisma, Supabase PostgreSQL, Redis, Upstash, Groq, Math.js, Zod, Tailwind CSS	End-to-end scaffold: FE shell, NestJS API, all agents wired, Redis & Validator live

Sprint 2 (May 4 - 16)	Integration & Core Workflow Hardening	Next.js, NestJS, Prisma, Redis, Groq, Server-Sent Events, NextAuth, Google OAuth, JWT, Jest, Supertest, Framer Motion	Reliable end-to-end: chat → agent pipeline → streamed response → visualization rendered
Sprint 3 (May 17-29)	MVP Refinement & Deployment Setup	Next.js, NestJS, Prisma, Redis, Groq, Server-Sent Events, NextAuth, Google OAuth, JWT, ESLint, GitHub, GitHub Actions, Vercel	Functional MVP: stable auth, improved response quality, CI/CD pipeline live
Sprint 4 (May 30 - June 11)	Stabilization & Integration Polish	Next.js, NestJS, Prisma, Redis, Groq, Server-Sent Events, NextAuth, Logging Middleware, Exception Filter, Jest, Supertest	Integrated prototype: hardened error handling, consistent agent context, demo-ready flows
Sprint 5 (June 12-25)	Final QA & Demo Readiness	Next.js, NestJS, Prisma, Supabase PostgreSQL, Redis, Groq, NextAuth, Jest, Supertest, ESLint, GitHub Actions, Vercel	Stable demo-ready build: zero blockers, rehearsed flow, seeded data, verified env vars

SECTION 6: SCALABILITY & REGIONAL RESILIENCE

- **Scaling Potential:** Socratix is designed to grow beyond a single institution. The multi-agent pipeline, particularly the Prompt Builder and Response Generator, can be adapted to support different ASEAN math curricula by tuning the prompt templates without changing the core architecture. Language support can also be expanded through prompt-level tuning, allowing Socratix to guide students in their native language while maintaining the same Socratix questioning structure and mathematical rigor. This means students can experience the same quality of guided reasoning in their own language, removing language as a barrier to deep mathematical understanding. As Socratic grows, this adaptability positions it as a scalable foundation for inclusive, inquiry-based math education across the entire ASEAN region.
- **Technical Constraints:** As a current prototype, Socratix has three key limitations. First, the visualization engine is still being developed and will be refined to support more native and complex math representations in future sprints. Second, the system depends on Groq API token limits, which may affect response availability under high concurrent usage. Third, Socratic is currently web-only, which limits accessibility for students in regions with limited desktop access. These constraints reflect the current stage of the prototype and represent a clear and intentional future development roadmap that extends beyond the hackathon timeline.